

Practical: Hopfield Network Storing 4 Vectors

Aim

To write a Python program to design a Hopfield Network which stores 4 vectors.

Objective

To study the working of Hopfield Neural Network and understand associative memory concepts.

Important Theory for Practical Writing

Hopfield Network is a recurrent neural network developed by John Hopfield in 1982.

It is mainly used for:

- Associative memory
- Pattern recognition
- Optimization problems
- Noise correction

Hopfield Network stores patterns and recalls them when noisy or incomplete input is provided.

It contains:

- Single layer of neurons
 - Fully connected architecture
 - Symmetric weights
 - No self-connections
-

Discrete Hopfield Network

Discrete Hopfield Network uses:

- Binary values:
 - 0 and 1OR
- Bipolar values:
 - +1 and -1

This program uses bipolar values.

Important Properties

1. Fully connected network
2. Symmetrical weight matrix

3. No self-feedback connection
 4. Uses iterative updating
 5. Works as associative memory
-

Weight Conditions

The network weights follow:

$$w_{ij} = w_{ji}$$

and

$$w_{ii} = 0$$

Where:

- w_{ij} = connection weight
 - w_{ii} = self-connection
-

Applications

1. Pattern recognition
2. Image restoration

Code

Write a python program to design a Hopfield Network which stores 4 vectors

```
import numpy as np
```

```
# Define the 4 vectors to be stored
```

```
vectors = np.array([
    [1, 1, -1, -1],
    [-1, -1, 1, 1],
    [1, -1, 1, -1],
    [-1, 1, -1, 1]
])
```

```
# Calculate the weight matrix
```

```
weights = np.zeros((4, 4))
```

```

for i in range(4):
    for j in range(4):

        if i == j:
            weights[i, j] = 0

        else:
            weights[i, j] = np.sum(vectors[i] * vectors[j])

# Define the activation function (in this case, a sign function)
def activation(x):
    return np.where(x >= 0, 1, -1)

# Define the Hopfield network function
def hopfield(input_vector, weights):

    output_vector = activation(np.dot(weights, input_vector))

    return output_vector

# Test the Hopfield network with one of the stored vectors as input
input_vector = vectors[0]

output_vector = hopfield(input_vector, weights)

print("Input vector:")
print(input_vector)

print("Output vector:")
print(output_vector)

```